

Training Small Models That Actually Work

A repeatable SFT → DPO pipeline for a Qwen3-0.6B documentation summarizer, and what it revealed about measuring model quality

Field	Detail
Team	Outliers (Group 1) — Sean Cheung (team lead), Ahmed Fayaz Wahab, Yash Rana, Khirod Kumar Ardi
Sponsor	Industry sponsor (name withheld under NDA); work performed on the sponsor's internal LLM training platform, referred to throughout as "the platform"
Project	SLM (Small Language Model) training for the sponsor's AI-powered document-summarization capability
Period	February 2026 (team formation) – August 2026 (final showcase)
Report date	14 August 2026
Primary sources	SFT_DPO_Training_Report_EXPO.docx; Model_Runs_Updated.xlsx (47 logged runs); weekly progress reports; dataset/config manifests and build scripts

Confidentiality note — this document is anonymised

This project was carried out under a non-disclosure agreement. The sponsor organisation, its personnel, and the name and internal addressing scheme of its LLM training platform have been withheld from this report and are referred to only as "the sponsor", "the sponsor contact" and "the platform".

The same anonymisation was applied to every public-facing project artefact — the expo trifold, the 12-minute talk, and the stakeholder deck. Technical findings about the platform's behaviour are retained because they are the team's own observations and constitute a core deliverable; nothing here identifies the sponsor, names the product, or reproduces its proprietary interfaces.

Please preserve this anonymisation in any onward distribution.

Contents

Right-click and choose "Update Field" if the table below appears empty — Word populates it on open.

1. Introduction

1.1 The problem, and why it matters

Organisations that publish or consume large volumes of technical documentation face a mundane but expensive problem: somebody has to read each page and say what it is about. Doing this with a large commercial language model works, but it is costly per call, sends content to a third party, and gives the organisation no asset of its own at the end. The alternative — training a model small enough to run on hardware the organisation already owns — is attractive precisely because it is cheap and self-contained, but it is widely assumed to be either too hard or too low-quality to be worth attempting.

Team Outliers set out to test that assumption directly. The question was stated simply: can a very small model — one that fits on a single shared GPU — be trained to perform a real, narrowly defined job reliably, using a repeatable process rather than one-off expert intuition?

The wider stake is not the summarizer. Document summarization was chosen as a deliberately narrow first use case because it is concrete, cheap to evaluate, and small enough to iterate on constrained hardware. What the sponsor actually needs is the capability behind it: a pipeline that can turn out small, specialised models for many tasks — classification, extraction, structured generation, domain question-answering — at low cost and with predictable quality. Every result in this report is therefore reported for its transferable lesson as much as for its specific number.

1.2 The sponsor and what they needed

The sponsor is an industry partner whose name is withheld under a non-disclosure agreement; it is referred to throughout as **the sponsor**. A single sponsor contact represented the organisation and met the team weekly (Tuesdays, 12:00–12:30) for the duration of the engagement. The sponsor operates an **internal LLM training platform** — referred to throughout simply as the platform — which manages datasets, launches training jobs against a shared GPU, and reports evaluation results. The sponsor supplied the platform, the compute, onboarding documentation, and the problem statement; the team supplied the data, the configurations, the experiments, and the analysis.

The requirement as stated at the 5 June internal sync was an LLM-based workflow that accepts either a website link or an uploaded document or PDF and produces two outputs: a short summary of roughly 300–400 words, and a longer, structured, wiki-style expanded article. Two constraints were explicit and non-negotiable. First, the model had to be small — the sponsor named 130 million, 350 million, or at most 500 million parameters, with a stated preference for the smallest model that could still do the job. Second, training had to happen on the sponsor's own platform, behind a user-facing knowledge base into which content is uploaded.

Scope narrowed during delivery. **The delivered system produces a single faithful summary of 150 words or less; the long-form wiki-style article was not built.** This is a genuine reduction against the original brief and is treated as such in Section 6.3. Multimodal handling of PDFs containing images was flagged as a stretch goal at the outset and likewise remained out of scope.

1.3 What was built — in one paragraph

Summary of the delivered system

The team built and validated an end-to-end training pipeline that takes public technical documentation and produces a small, deployable summarization model. A hand-built data pipeline fetches documentation pages, strips navigation and code blocks, caps each page to a 4,096-token budget, and emits three dataset shapes: supervised demonstrations, preference triples, and good/bad labelled rows. On the sponsor's platform, LoRA adapters are trained over an 8-bit Qwen3-0.6B base — first with Supervised Fine-Tuning to teach the task, then with Direct Preference Optimization to sharpen quality — and the resulting adapters are evaluated against a frozen, shared held-out set. Across 47 logged training runs the SFT stage reached 100% pass on held-out summaries (25/25 and 65/65 on the two largest evaluation sets), and the DPO stage was tuned to a defensible production configuration captured in `dpo_config_FINAL.yaml`, with the strongest run reaching 93% held-out reward accuracy.

The most consequential output, however, was not a model: it was the discovery that the platform's default scored metric could not distinguish a good preference-tuned model from a bad one, together with a replacement metric derivable from data the platform already logs.

2. Architecture Overview and Key Details

2.1 System diagram

The system divides into four stages. Stages 1 and 3 ran on the team's own workstations; stage 2 is the sponsor's hosted platform; stage 4 exists as configured export targets rather than a live deployment. The diagram below is deliberately high-level — the trainer configurations, metrics and evaluation mechanics behind each box are set out in Sections 2.2 and 5.

System Architecture — Documentation Summarizer Training Pipeline

A small model is trained in two stages, checked on documents it has never seen, and adjusted until it is good enough to ship

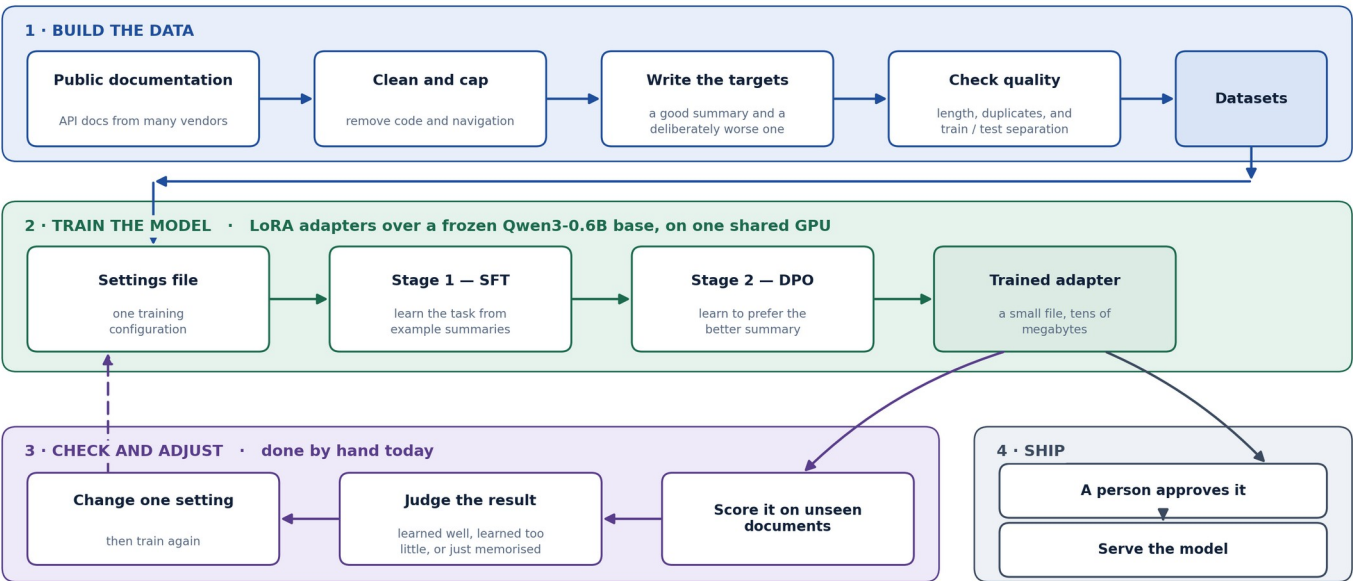


Figure 1 — End-to-end architecture. Data is built on the team's own workstations (band 1); training runs on the sponsor's platform against one shared GPU (band 2); the check-and-adjust loop (band 3) was performed entirely by hand and is the single largest candidate for automation. A third alignment stage, KTO, is configured but blocked by a library regression and is omitted here for clarity.

2.2 Tech stack

Layer	Technology	Role and specifics
Language	Python 3	All data-preparation tooling. Standard library only — argparse, json, re, random, statistics, collections, pathlib. No third-party dependencies and no requirements file, which keeps the scripts portable but means there is no pinned environment.
Base model	Qwen3-0.6B	~600M parameters, ~152k vocabulary. Loaded in 8-bit precision. Path /models/base-models/qwen3-0-6b, identical across every SFT and DPO config.
Adaptation	LoRA (PEFT)	Base weights frozen; small adapter matrices trained on top. Targets q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj. SFT: r=16, α=32, dropout 0.05. DPO final: r=8, α=16, dropout 0.10. Saved artefact is tens of megabytes.
Alignment	SFT → DPO → (KTO)	Hugging Face TRL under the platform's wrappers. SFT teaches the task from {prompt, completion}; DPO refines from {prompt, chosen, rejected}; KTO from {prompt, completion, label} — configured but blocked. A PPO config exists but is gated on a learned reward model.
Platform	Sponsor-hosted internal training platform	Dataset management, job scheduling, telemetry, evaluation harness and artefact storage. The configuration schema is a Pydantic model with extra="forbid", so unrecognised keys are rejected outright. Each run is addressed by an opaque run

Layer	Technology	Role and specifics
		identifier scoped to the owning user.
Compute	One shared 16 GB GPU	Single job at a time. Forces <code>per_device_train_batch_size = 1</code> with gradient accumulation, gradient checkpointing, <code>bf16</code> , <code>paged_adamw_8bit</code> , and the 4,096-token sequence cap adopted across the SFT and DPO configurations.
Evaluation	Built-in embedding scorer	Cosine similarity of generated summary to reference; pass threshold 0.75, review threshold 0.55; <code>max_new_tokens</code> 320. An exact-match scorer also exists but is unsuitable for free text.
Export	Merged model + Ollama	<code>produce_merged_model: true</code> writes a standalone merged model alongside the LoRA adapters; an Ollama output directory is configured in every SFT and DPO config.
Data sources	Public vendor API docs	OpenAI, Llama, DeepSeek (SFT); Google Gemini, Firecrawl (DPO); Cohere, xAI, Perplexity, Anthropic, Qdrant, Stripe, Supabase, Docker, Twilio, Sentry, Resend (KTO); Stripe, GitHub, Slack, AWS and others in the expanded held-out set.
Tracking	Google Sheets + Excel	A four-tab workbook (Overview / SFT / DPO / KTO) is the system of record for all 47 runs. <code>report_to: [none]</code> in every config — no MLflow, Weights & Biases or TensorBoard integration was enabled.

2.3 How the components interact — data flow from input to output

The pipeline is linear with one feedback loop. Following a single document from source to trained adapter:

- Source selection.** URLs are maintained by hand in numbered lists inside `sft_README.md`, `dpo_README.md` and `kto_README.md`. These README files are the pipeline's input contract: the build scripts parse the lists directly out of them — although `build_dpo_dataset.py` hard-codes the filename `README.md`, which does not match the delivered `dpo_README.md`, so the script as shipped cannot locate its own input.
- Fetch and extract.** Each page is retrieved and reduced to its article body — from the single top-level heading to the end — discarding the surrounding site navigation. Vendor documentation is JavaScript-rendered and carries a large repeated sidebar, so raw payloads typically run 50–100 KB.
- Clean.** Fenced code blocks, line-number gutters, navigation bullets and reference tables are stripped. These contribute heavy token count and almost nothing to a summary, and they were a direct contributor to the original out-of-memory failure.
- Cap.** The cleaned article is truncated at a line boundary to fit a 4,096-token budget with room for the generated summary. Because the target summary lives in a separate field, capping the prompt can never remove the training label. Maximum observed prompt length after capping was ~3,466 tokens.
- Generate targets.** For SFT, a ≤150-word summary. For DPO, a chosen summary plus a deliberately degraded rejected summary drawn from a weighted six-mode defect taxonomy — factual error, wrong scope, vague/generic, key omission, verbose padding, and leaked reasoning. For KTO, a single completion with a true/false label.
- Quality gates.** Word-count and reasoning-tag checks, de-duplication, and a guaranteed train/eval URL disjointness check. Records are written to JSONL; randomness is seeded so datasets regenerate identically.
- Upload and launch.** JSONL files are uploaded to a per-run platform workspace. A YAML config is authored and submitted; the platform validates it against its Pydantic schema, queues the job, and runs it on the shared GPU.
- Train.** SFT trains LoRA adapters over the frozen 8-bit base. DPO then initialises from those same SFT adapters (`initial_adapter_path`) and trains against a frozen copy of the SFT checkpoint as its reference model, with beta constraining how far the policy may drift.

9. **Evaluate.** The platform generates a summary for each test document and scores it with the embedding scorer, writing per-record results. Training telemetry — loss, reward accuracy, reward margin, KL divergence — is written to the Progress view.
10. **Analyse and adjust (the feedback loop).** The team exports per-record margins from the evaluation logs, computes held-out reward accuracy by hand, classifies the run as underfit, healthy or overfit using the train-versus-held-out gap, changes exactly one lever, and relauches. This loop was performed entirely manually and is the single largest candidate for automation.
11. **Export.** The winning run produces LoRA adapters plus a merged standalone model, with an Ollama export directory configured for serving.

The most important architectural constraint

Every structural property of this system follows from one fact: a single shared 16 GB GPU that runs one job at a time. It caps model size, forces a batch size of one, dictated the $32,768 \rightarrow 4,096$ sequence-length reduction, and turns what should be a parallel hyperparameter sweep into a long series of sequential launch-wait-interpret cycles. Roughly half the bottlenecks in Section 8 are downstream of it.

3. Development Process

3.1 Methodology

The team did not run formal Scrum. No project artefact uses the words sprint, standup, retrospective, backlog or story point, and no sprint board was maintained. What operated instead was a lightweight, cadence-driven process built around three commitments — one set by the charter, two agreed at the 5 June internal sync:

- **A weekly sponsor checkpoint.** Tuesdays, 12:00–12:30, with the sponsor contact over Google Meet, established in the 20 May charter. This was the only fixed external commitment and it anchored the reporting rhythm.
- **An internal cadence of three touchpoints.** Monday in-person check-in and progress review; Wednesday in-person working session for model training; Friday online update. Each member was expected to be able to explain their own work in a 3–5 minute update at the Monday check-in.
- **A written weekly progress report.** A fixed template — Completed / In progress / Next week's priorities / Challenges and blockers / Questions and support needed. Ahmed proposed rotating authorship to share the load; in practice all but the first surviving report carry Sean's byline.

Task division was deliberate and, in retrospect, methodologically fortunate. Rather than partitioning the work by component, Ahmed proposed that **all four members build the first summarization task independently** so that everyone learned the platform, splitting work only for the second task; each person would create their own project and consolidate later. Applied to the preference-tuning stage, this produced three genuinely different search strategies running in parallel against the same problem:

Member	Role	Contribution
Sean Cheung	Team lead; disciplined single-lever search	Owned documentation and project structure; authored the weekly reporting stream, the dataset and config manifests, the run ledger and the final reports. Ran a controlled one-lever-at-a-time DPO experiment (Exps 1–6) that isolated which levers actually mattered. 20 SFT and 11 DPO runs.
Ahmed Fayaz Wahab	Scope definition; SFT run orchestration	Clarified the requirement with the sponsor and pushed for a practical task split. Orchestrated supervised training job runs across the team — launching, monitoring failures, relaunching — which is why the OOM and reasoning-leak failures were worked through quickly. Owns no rows in the run ledger; his contribution is operational rather than experimental.
Yash Rana	Larger datasets; bolder settings	Ran fewer, more aggressive experiments on the largest datasets. 7 SFT runs (up to 250/65) and 2 DPO runs — one producing the clearest overfitting case in the programme (63% held-out) and one producing the strongest result of anyone (93% held-out).
Khirod Kumar Ardi	Broad configuration sweep	Ran a wide search across beta 0.1–0.5, LoRA rank 4–16 and both sigmoid and IPO losses. 2 SFT and 5 completed DPO runs, all landing in the healthy band (74–79% held-out), which established that a wide range of settings works when data and regularisation are balanced.

Table 1 — Team roles as evidenced by the run ledger and project artefacts.

The experimental method

The project's defining methodological choice was not organisational but experimental: change exactly one lever per run, from a fixed moderate baseline, so that every observed effect is attributable to a known cause. This was formalised into a five-run lever-test plan with five pre-generated config YAMLS, and into an explicit triage decision table that the team applied consistently and later proposed as the rule set for an automated tuner:

- Reward accuracy stuck at 0.50 → lower beta (the model is on too tight a leash to move).

- Reward accuracy racing to 1.00 while held-out accuracy drops → raise beta, cut epochs, or reduce adapter rank.
- KL divergence exploding → lower the learning rate.
- Always change one lever per run; rank candidates on held-out reward accuracy, never on the saturated pass-rate.

A second methodological commitment made the three parallel searches comparable: a **frozen, shared evaluation set**. A canonical 60-record held-out file was seeded from Sean's dpo_eval_60.jsonl, published with a manifest instructing the team to freeze it, and later verified byte-identical against Yash's evaluation set — all 60 prompts and all 60 chosen answers matched, across 43 unique documents. Without that verification the cross-team comparison in Section 5 would have been meaningless.

3.2 Timeline and milestones

Date (2026)	Milestone
6 Feb	Team Outliers (Group 1) formed; skills and preferences captured in the group information sheet.
20 May	Project charter created for the sponsor's small-language-model training engagement; weekly Tuesday 12:00–12:30 sponsor sync established.
w/c 1 Jun	Direction agreed with the sponsor; access granted to the ML platform and onboarding documentation. Team begins reviewing the platform documentation.
5 Jun	Internal sync fixes the requirement (URL / document / PDF → short summary + wiki article), the 130M–500M model-size constraint, the Mon/Wed/Fri cadence, and the "everyone builds task one" work-breakdown.
8 Jun	First dataset artefacts generated (ai_api_summarization_*.jsonl — 100 train / 20 eval, chat-message format).
16 Jun	SFT dataset report and sft_config.yaml finalised; Model_Training_Pipelines and Custom_Model_Feasibility_Plan prepared. SFT out-of-memory failure diagnosed and fixed via the 32,768 → 4,096 sequence cap.
19–21 Jun	DPO dataset design published (dpo_README.md); build_dpo_dataset.py written; SFT v3 train/eval sets (100/25) built across OpenAI, Llama and DeepSeek documentation.
3 Jul	DPO Run 2 (79/19/30) — 76.7% held-out but training reward accuracy 1.00: first overfit signal. Platform schema rejections (Pydantic extra="forbid") documented and worked around.
5 Jul	DPO Run 7 (89/9/30) — 73.3% held-out with a runaway margin; the overfitting analysis that set the programme's direction. Preference set expanded 98/30 → 200/34. KTO dataset and config built end-to-end.
6–12 Jul	Run 8 (v2 remediation) — 79.4% held-out but underfit. Five-run lever plan and five config YAMLS authored. Six-run lever experiment executed; the evaluation-metric problem diagnosed; dpo_config_FINAL.yaml produced.
10 Jul	Run-log cross-reference completed: every run identifier mapped to its platform record; two data-integrity defects in the ledger identified and documented.
13 Jul	Shared 60-record evaluation set frozen and published with a manifest. Yash's runs logged. Four-tab run workbook, ~20-page consolidated report and a 10-slide stakeholder deck delivered. KTO platform blocker isolated to a TRL regression. Text-to-speech research foundation scoped as a follow-on workstream.
5 Aug	Consolidated SFT/DPO training report issued.
12–13 Aug	Automation vision documents produced; expo trifold designed, exported and print-ready; talk track written.
14–15 Aug	13-slide, 12-minute showcase deck and word-for-word speaker script completed; expo edition of the training report finalised.

Table 2 — Project timeline. Note the reporting gap between the week of 1 June and 5 July: no weekly updates survive for that five-week window, which coincides with the SFT dataset rebuild and the first DPO runs.

3.3 Collaboration and project-management tooling

This section is reported strictly against evidence in the project artefacts, because it is an area where it would be easy to overstate.

Tool	Status	Use
Google Meet	Used	Weekly sponsor sync and the Friday internal online update.
Google Drive	Used	The single system of record for the entire project — charter, weekly updates, planning documents, datasets, configs, scripts and reports all live in one shared Drive folder.
Google Sheets	Used	The live "Model Runs" sheet, cited as a source in most weekly updates.
Microsoft Excel	Used	Model_Runs_Updated.xlsx — the four-tab consolidated run ledger (Overview / SFT / DPO / KTO) covering all 47 runs.
Sponsor training platform	Used	Doubled as the experiment tracker: per-run configuration, logs, progress telemetry, evaluation results and artefacts, each addressed by an opaque run identifier.
Microsoft Word / PowerPoint	Used	Reports, weekly updates, speaker script and the 13-slide showcase deck.
Figma Make	Used	Generated the 48×36-inch expo trifold from seven documented sequential prompts.
Gamma	Used	Generated the 10-slide stakeholder deck distilled from the training report.
GitHub	Discussed only	Proposed on 5 June for technical files, markdowns and training artefacts. No repository, .gitignore, requirements.txt or CI configuration exists anywhere in the project folder.
Microsoft Teams	Discussed only	Proposed on 5 June for document storage; Google Drive was used instead.
Jira / Trello / Confluence / Slack / Asana	Not used	A recursive search of every text artefact returns no project-management usage of any of these. (Khirod lists several on his skills profile, but that is a personal résumé line, not project usage.)

Table 3 — Collaboration tooling, reported against artefact evidence.

The honest assessment is that the team ran a document-centric process, not an engineering process. Version control existed only as filename conventions — `sft_config.yaml` / `sft_config_v2.yaml`; `dpo_config.yaml` → `_clean` → `_v2` → `_FINAL`; six numbered experiment configs — and the run ledger was maintained by hand. This was workable at four people and 47 runs, and it did produce a complete, auditable trail. It would not scale, and it produced two concrete defects: duplicated and corrupted run identifiers in the ledger (one with a stray carriage return, one truncated, one that would not resolve), and a Training Loss column that silently mixes two different metrics across DPO rows. Both were caught by a manual cross-reference pass on 10 July, which is exactly the kind of check a repository and a schema would have made unnecessary.

4. Data Handling and Ethics

4.1 Where the data came from

No client data was used, and no personal data was used. Every training record in this project derives from publicly accessible vendor API documentation, crawled by the team. The reference summaries and the deliberately degraded rejected summaries are synthetic — model-generated and then validated against explicit rules. In the taxonomy the brief asks about, the corpus is therefore **scraped public documentation with synthetic targets**.

Stage	Format	Sources	Size	Disjointness
SFT	{prompt, completion}	developers.openai.com (×70), llama.developer.meta.com (×21), api-docs.deepseek.com (×34)	100 train / 25 eval	Train and eval URL sets verified unique and non-overlapping
DPO	{prompt, chosen, rejected}	ai.google.dev/gemini-api (×89), docs.firecrawl.dev (×41)	200 train / 60 held-out	26 of the 60 held-out records are deliberately out-of-domain (Stripe, GitHub, Slack, AWS and others) to test transfer rather than recall
KTO	{prompt, completion, label}	Cohere, xAI, Perplexity, Anthropic, Qdrant, Stripe, Supabase, Docker, Twilio, Sentry, Resend — 11 domains	200 train / 36 eval	Explicitly non-overlapping with both the SFT sources and the DPO source

Table 4 — Dataset provenance. Source families were kept disjoint across stages by design, so that a model tuned on Gemini documentation is evaluated partly on documentation it has never seen from entirely different vendors.

Two details of the data design deserve emphasis because they are what makes the preference signal meaningful rather than decorative:

- **The rejected summaries are engineered, not merely worse.** Six labelled defect modes are generated programmatically with fixed weights — factual error (weight 30, mutating integers or flipping GET→POST, required→optional, bearer token→API key), wrong scope (22, substituting a neighbouring document's summary), vague/generic (18), key omission (14, keeping only the first third of sentences), verbose padding (8), and leaked reasoning (8). Because every rejection carries its defect label, it is possible to ask which failure mode the model learned to avoid — not just whether it improved.
- **Generation is seeded and reproducible.** `random.seed(42)` in the v2 builder and `random.seed(7)` in the v3 assembler, with fixed shuffle seeds throughout, so any dataset can be regenerated byte-identically from the same URL lists.

4.2 Storage and security

Project documents, datasets, configurations and scripts are stored in a single shared Google Drive folder under the team's institutional accounts. Training data and artefacts additionally live in isolated per-run workspaces on the sponsor's platform, each holding the input datasets on one side and the trained adapters, merged weights and Ollama exports on the other.

Three security-relevant settings were applied consistently across every training configuration, and are worth recording because they were deliberate:

- `local_files_only: true` — the trainer is not permitted to reach out to a model hub at run time.
- `trust_remote_code: false` — no arbitrary model-supplied code is executed during loading.

- `hf_token_env`: `HF_TOKEN` — credentials are supplied by environment variable, never written into a configuration file. No secret appears in any committed artefact.

The one confidentiality control that was actively enforced was sponsor anonymisation in public-facing materials. The expo board reviewer checklist includes the explicit item "No sponsor name and no platform/product name anywhere on the board," the talk track instructs presenters to say only "our sponsor," and the prepared Q&A answer for the inevitable question is scripted: the sponsor is an industry partner building an internal model-training platform, and the names cannot be shared. This was applied consistently across the trifold, the deck and the 12-minute talk.

4.3 Consent, privacy and bias considerations

This section is where the project is weakest, and it is more useful to say so plainly than to dress up the planning documents as controls.

Consideration	Position
Personal data	None knowingly present. The corpus is vendor API documentation — endpoint references, parameter tables, developer guides — which does not ordinarily contain personal information. No PII detection or redaction pass was actually run to confirm this.
Consent	Not applicable in the usual sense: no human subjects, no user data, no client records. No consent framework was required and none was built.
Licensing and terms of service	A genuine gap. The corpus consists of copyrighted vendor documentation crawled without a recorded review of each publisher's terms of service or usage rights. The Corpus Formation Plan lists "confirm licensing / usage rights for every source (commercial use, redistribution, attribution)" as a required gate — though that plan was scoped to a from-scratch pretraining corpus, and the item remains an unchecked checklist entry rather than a completed activity. For an academic proof of concept this is defensible; for the sponsor's production pipeline it is a prerequisite, not an optional step.
Bias and fairness	No bias or fairness evaluation of the summarizer was performed. The nearest thing to a fairness control is a structural one: source families were deliberately kept disjoint across stages, and 26 out-of-domain records were added to the held-out set specifically to detect a model that had learned one vendor's house style rather than the task. That measures domain generalisation, not demographic or representational bias.
Data quality and contamination	Actively controlled. Train/eval disjointness is enforced programmatically and verified. One residual weakness is documented honestly by the team: 17 of the 60 held-out documents appear twice, so they carry roughly double weight in the score; a de-duplicated 43-record variant was produced but the 60-record set remained canonical for comparability.
Planned but not implemented	The Corpus Formation Plan specifies a PII and safety stage (Microsoft Presidio for redaction, NSFW and toxicity filtering), decontamination against a golden set, and provenance tracking with versioned manifests. That plan was written for a from-scratch pretraining corpus that was never built, so these gates were never formally in scope for the fine-tuning corpus — but none of them were implemented there either. They are correctly scoped and should be treated as a required work item for any successor team, not as a completed capability.

Table 5 — Ethics and data-governance position, reported honestly against what was actually implemented.

4.4 Disclosure: AI tools used during development

AI assistance was central to this project, not incidental, and it was used in a way that materially shaped the dataset. The disclosure below is complete to the best of the team's records.

Tool	How it was used
Claude (Anthropic)	The primary development assistant, and — importantly — a data-generation participant. The workflow is documented explicitly in <code>build_dpo_dataset.py</code> : "in this project the crawl + summary-writing is performed interactively (Claude fetches and cleans each page, then writes the chosen/rejected text). This script is the

Tool	How it was used
	deterministic scaffold for the mechanical parts." The operating pattern was README-driven: the team pasted URL lists into a README, and Claude fetched each page, cleaned it, generated the chosen and deliberately degraded rejected summaries, and wrote the JSONL. Claude also assisted with the build scripts, configuration authoring, run-log analysis and report drafting.
Figma Make	Generated the 48×36-inch expo trifold poster from seven sequential prompts, which are preserved verbatim in Trifold_FigmaMake_PromptLog.md.
Gamma	Generated the 10-slide stakeholder deck distilled from the consolidated training report.
The trained model itself	SFT reference summaries were model-generated and then validated against the length, content and cleanliness rules — not independently fact-checked line by line. The SFT dataset report states this limitation directly and recommends a human spot-check before relying on the model in production.
The honest implication of AI-generated training data Because the reference summaries were model-generated and validated by rule rather than by human fact-check, the ceiling on the trained model's factual quality is set partly by the generating model's quality. The team documented this rather than glossing it, and recommended human spot-checks. It also cuts the other way, and this is the more interesting point: the AI-assisted crawl-clean-generate loop is precisely the workflow the proposed data-sourcing agent would automate. The project did not merely use AI to build a dataset — it prototyped, by hand, the agent it is now recommending be built.	

5. Key Accomplishments

5.1 Supervised fine-tuning: solved and reliable

The SFT stage is complete and dependable. Its story is not about hyperparameters — the configuration barely changed after June — but about diagnosing two concrete failures and fixing both in the data and the prompt rather than the model.

Run (owner)	Train/Val/Test	Passed	Eval loss	Note
599040fb (Sean)	80 / 20 / 25	25/25 (100%)	1.756	Best Sean SFT run
9a8f8aee (Sean)	40 / 10 / 20	19/20 (95%)	1.237	Best 40-record run
b2d5fc6c (Yash)	250 / 50 / 65	65/65 (100%)	not logged	Largest set; clean pass
6cb5d1b7 (Yash)	250 / 50 / 65	65/65 (100%)	0.379	Newer Run 3; eval loss now captured
cdce1e9a (KhiroD)	80 / 20 / 25	25/25 (100%)	1.759	r=8 configuration

Table 6 — Representative SFT results. Across 29 logged SFT runs, recent configurations pass 24–25 of 25 held-out summaries, and the 250-record runs pass 65/65.

Two named failures were diagnosed and eliminated:

- **Out-of-memory, fixed by data rather than by shrinking the model.** At max_seq_length 32,768 the loss computation materialises a tensor of roughly sequence × 152k vocabulary — about 20 GB on its own, exceeding the entire GPU. The model was never the problem; Qwen3-0.6B is tiny. Cleaning the documents and capping to 4,096 tokens cured it permanently — a cap adopted across Sean's and KhiroD's configurations, though one later Yash run is logged at 32,768 against already-capped data. Generation length was cut from 32,768 to 320 tokens for the same reason — a 150-word summary needs roughly 200.
- **Leaked reasoning, fixed in the prompt.** Qwen3 emits a <think> block by default and was spending its entire output budget on a hidden reasoning trace instead of producing the summary. An empty assistant prefix plus a /no_think directive suppressed it. A third, smaller fix: save and eval steps were reduced from 25 to 10, because on the small dataset the run never reached step 25 and therefore never wrote a checkpoint or an eval score.

5.2 Preference tuning: a tuned, defensible production configuration

Sean's controlled experiment changed exactly one lever per run from a moderate baseline, making every effect attributable. The result is unusually clean, and unusually negative in a useful way: across six controlled runs spanning four levers, only two levers did anything at all.

Experiment	Lever changed	Train reward acc	Train margin	Verdict
Baseline	beta 0.15, r8, 1 epoch	0.50	−0.11	Underfit — the policy never moved
Exp 2	beta 0.10	0.75	+0.07	First run to learn anything
Exp 3	r16 at beta 0.15	0.50	−0.16	Underfit — added capacity did not help
Exp 4	beta 0.10 + 2 epochs	0.75	+0.44	WINNER — margin multiplied ~6×
Exp 5	IPO loss at beta 0.10	0.50	−0.15	No traction
Exp 6	r16 at beta 0.10	0.50	−0.05	Underfit — added capacity did not help

Table 7 — The single-lever sweep. beta 0.15 is a dead zone; beta 0.10 is the threshold at which the policy can move at all; a second epoch is the single most effective change made in the entire programme.

The selected production configuration, promoted to `dpo_config_FINAL.yaml`: beta 0.10, sigmoid loss, LoRA $r=8$ / $\alpha=16$ / dropout 0.10, 2 epochs, weight decay 0.01, learning rate $5e-6$ (roughly 40× lower than SFT's $2e-4$), continued from the SFT adapters. It records reward accuracy 0.7500 and reward margin +0.4401 — squarely inside the healthy band, with the chosen reward rising and the rejected reward falling, which is the signature of a model actively separating good from bad rather than merely becoming more confident.

5.3 The pivotal finding: the default metric could not tell good from bad

The result that reframed the project

The platform's default scored metric — an embedding similarity pass-rate — stayed pinned at 0.79–0.82 across the lever-sweep runs on the fixed evaluation set, regardless of what changed. In a direct contrast, the clearly underfit baseline scored 81.7% while the tuned winner scored 80.0%: the metric ranked the worse model higher. Expanding the evaluation set from 34 to 60 records did not fix it, because the problem was never sample size.

The reason is structural and, in hindsight, obvious. DPO is designed to keep the policy close to the SFT reference — that is what beta does. So a preference-tuned model produces near-identical text to an untuned one, and a metric that compares text to a reference cannot see the difference. The metric is dominated by the SFT base and blind to the preference improvement DPO exists to deliver.

The fix required no new instrumentation. Held-out reward accuracy — the fraction of unseen preference pairs where the model scores the chosen summary above the rejected one — is computable from per-record margins the platform already writes to its evaluation logs. It simply is not surfaced as the scored, gated metric. Recomputing every run on that uniform basis turned a flat line into a 30-point spread.

5.4 Cross-team results on the uniform held-out metric

Run	Train reward acc	Held-out acc	Reading
Sean — winner (Exp 4)	0.75	79–84%	Healthy; small train-vs-held-out gap
Sean — overfit (Runs 2 / 7)	1.00	73–78%	Memorised the training pairs; held-out flat
Yash — DPO on 304 pairs	1.00	63%	Severely overfit: a 37-point generalisation gap
Yash — DPO on 200 pairs (later)	0.81	93%	Strongest held-out result in the programme; no gap
Khiron — five-run sweep	0.75 (1 of 5 retained)	74–79%	Healthy across beta 0.1–0.5, r 4–16, sigmoid and IPO

Table 8 — All runs re-scored on one uniform held-out metric. The comparison is only legitimate because the evaluation files were verified identical.

The two Yash runs are the most instructive pair in the project, because **they share the same hyperparameters and produce opposite outcomes**. Same beta (0.25), same rank (8), same two epochs. The first, trained on the largest preference set (304 pairs), memorised its training data — reward accuracy 1.00, margin +6.9 — and generalised worst of anyone at 63%. The second, retrained from a fresh SFT checkpoint on a cleaner 200-pair set, reached 93% held-out (peaking at 0.95) with a large held-out margin of +1.75 and essentially no gap to its 0.81 training accuracy. The only things that changed were the adapter it started from and the quality of the preference data.

That comparison would be worthless if the two runs had been scored on different data, and the team checked. Yash's 60-record held-out set was verified **byte-identical** to Sean's `dpo_eval_60` — all 60 prompts and all 60 chosen answers match, across the same 43 unique documents — so 93% and 84% are measured on exactly the same material and are directly comparable. The one axis that is not comparable is the raw reward margin, which is inflated by the higher beta; accuracy is beta-invariant and remains the fair comparator. This is the kind of verification that separates a real result from a plausible one.

5.5 Quantified summary

Achievement	Measure
Total training runs executed and logged	47 (29 SFT, 18 DPO) across three contributors, each traceable to a platform run record
SFT pass-rate on the largest held-out set	65/65 (100%); 25/25 (100%) on the standard 25-record set
Best held-out reward accuracy achieved	93% (peak 0.95), with no train-versus-held-out gap
Tuned production configuration	dpo_config_FINAL.yaml — reward accuracy 0.7500, margin +0.4401, inside the healthy band
Levers conclusively tested	Six controlled runs across four levers; two found to matter (beta, epochs) and two shown not to help (adapter rank, IPO loss)
Evaluation-metric defect identified	Default pass-rate flat at 0.79–0.82 across the lever sweep; ranked the underfit model (81.7%) above the winner (80.0%)
Replacement metric delivered	Held-out reward accuracy, computable from existing logs, producing a 30-point spread where the default produced none
Evaluation integrity verified	60/60 prompts and chosen answers confirmed identical across two contributors' eval files, across 43 unique documents
Dataset scale reached	SFT 100 train / 25 eval across 3 vendor domains; DPO 200 train / 60 held-out with 26 out-of-domain records; KTO 200/36 across 11 domains
Out-of-memory failure	Root-caused to sequence length × vocabulary interaction and permanently eliminated

Table 9 — Quantified outcomes.

5.6 What the sponsor specifically cared about

- A model small enough to be practical.** Substantially met, with one deviation worth stating plainly: Qwen3-0.6B is roughly 600M parameters, just above the sponsor's stated 500M ceiling, and no artefact records that being agreed. The practical intent was met regardless — the trained artefact is a LoRA adapter of tens of megabytes over a frozen shared base, so the marginal cost of an additional specialised model is negligible, and multiple task adapters can serve from the same 16 GB GPU.
- Proof that the platform can produce a working model end to end.** Delivered: SFT and DPO both run reliably, with 47 documented runs and a promoted final configuration.
- Actionable feedback on the platform itself.** Arguably the highest-value deliverable. The team returned a prioritised, evidence-backed list of platform defects: the non-discriminating scored metric, silent schema rejections, telemetry retained only for the most recent run, and a TRL version regression that blocks KTO entirely. Each is tied to a specific run and a specific cost in engineering time.
- A path to scale.** A four-phase roadmap with a stated sequencing rationale, plus two concretely specified agents, each of which automates a loop the team has already executed by hand.

6. Insights and Outcomes

6.1 What the team learned technically

What worked

- **Changing one lever at a time.** Every clean conclusion in this report comes from a single-variable change. The one earlier configuration revision that bundled several changes together produced results that could not be attributed to anything, and had to be re-run.
- **Regularisation over ambition on small data.** The winning configuration is the conservative one: low rank, high dropout, weight decay, a tight-but-not-strangling leash. The worst generalisation in the programme came from the most aggressive settings applied to the largest dataset.
- **Diagnosing from signals rather than guessing.** The OOM was traced to a tensor-size calculation, not to a hunch about model size. The reasoning leak was traced to the output trace. The overfitting was traced to a reward decomposition showing the margin came from pushing rejected rewards down to -1.15 rather than lifting chosen rewards above $+0.40$. Each diagnosis pointed at a specific fix.
- **Freezing a shared evaluation set — and then verifying it.** Publishing a canonical 60-record file with a manifest, and later confirming byte-identity across contributors, is what made the cross-team comparison defensible rather than anecdotal.

What surprised them

- **The metric was the problem, not the model.** The team spent weeks tuning against a scoreboard that could not see what they were tuning. Discovering that the default metric ranked the underfit model above the winner was the moment the project's centre of gravity shifted from modelling to measurement.
- **Identical settings, opposite outcomes.** Two runs with the same beta, rank and epoch count produced 63% and 93% held-out accuracy. The difference was the starting checkpoint and the cleanliness of the preference pairs. The uncomfortable corollary — stated plainly in the team's own talk track — is that on small data, **the dataset is the hyperparameter**, and the knobs tuned for weeks were never the binding constraint.
- **More data made things worse, not better.** The single most aggressive run had the largest dataset (304 pairs) and generalised worst. Data volume without matching regularisation is not an improvement.
- **Held-out accuracy barely moved across the entire lever sweep.** Roughly 0.78–0.84 for every run. This is not a disappointing result — it is the evidence that the ceiling is data scale rather than tuning skill, and it is why the roadmap leads with data rather than with hyperparameters.
- **Memory was governed by vocabulary, not parameters.** On a memory-constrained GPU, sequence length interacting with a $\sim 152k$ vocabulary during loss computation dominates memory more than the model weights do. This generalises well beyond this project.

6.2 What the sponsor gained

Four things, in ascending order of durability:

12. **A working summarizer and a validated configuration.** An SFT stage that passes 100% of held-out summaries on the largest evaluation set, and a DPO configuration promoted to a named production file with recorded, healthy training metrics.
13. **A mapped configuration space.** Three independent search strategies — disciplined single-lever, aggressive large-dataset, and broad sweep — collectively map the DPO space more completely than any one of them could. The sponsor now knows which levers matter (beta, epochs), which do not (adapter rank, IPO loss), and where the failure boundaries sit.

14. **A prioritised platform defect list with evidence.** Not opinions — each item is tied to a run, a log, and a cost. The scored metric does not discriminate. The config schema rejects keys silently. Telemetry is retained only for the most recent run, which forced manual reconstruction of cross-run comparisons. A TRL regression blocks KTO outright. These are actionable engineering tickets.
15. **A designed, sequenced automation roadmap.** Two agents — a data-sourcing agent and an auto-tuning agent — each specified against a loop the team has already executed manually, plus the enabling foundations and the reasoning for the order in which to build them. The sponsor's stated goal was a training capability, not a summarizer, and this is the part of the deliverable that speaks to it.

6.3 Pivots made along the way, and why

Pivot	From → To	Why
Scope reduction	300–400 word summary plus a long-form wiki-style article → a single ≤ 150 -word summary	The narrower task is concrete, cheap to evaluate, and small enough to iterate on one shared GPU. It became the testbed for the pipeline rather than the product. This is a real reduction against the original brief; the long-form output was never built, and no artefact records a formal decision to drop it.
Sequence length	32,768 → 4,096 tokens	Forced by the out-of-memory failure. The loss computation at 32k materialises a ~ 20 GB tensor against a 16 GB GPU.
SFT corpus	4 Gemini pages (with train and eval overlapping) → 50/20 OpenAI → 100/25 across OpenAI, Llama and DeepSeek	Six documented iterations, each driven by an observed failure rather than by guesswork: no true held-out set, then memory pressure, then insufficient domain diversity.
Preference dataset	6-pair placeholder → 98/30 → 200/34 → 200 train / 60 held-out	Driven by the overfitting analysis of Run 7. The diagnosis concluded that the root cause was data scale and weak pair contrast, not insufficient training — so the response was to expand and sharpen the data and to regularise, rather than to train harder.
The comparator metric	Embedding pass-rate → held-out reward accuracy	The pivotal pivot. The default metric could not separate a good model from a bad one; the replacement was derived from per-record margins already present in the evaluation logs.
Adapter configuration	$r=16$, dropout 0.05 → $r=8$, dropout 0.10	Less capacity and more regularisation, in direct response to the overfitting diagnosis.
Third alignment stage	KTO planned and configured → shelved	A library-version regression: the platform passes a tokenizer argument that the installed TRL KTOTrainer no longer accepts. A version-pinning and CI problem, not a modelling one.

Table 10 — Pivots. Note that five of the seven were triggered by an observed failure signal rather than by a change of mind, which is the pattern the team explicitly set out to follow.

7. Deployment Strategy

7.1 Current state: export-ready, not deployed

To be direct: **the model is not deployed and there is no live inference endpoint**. What exists is a fully configured export path, exercised on every run, plus a clear picture of what production would require. That is an honest place for a capstone to land, and the gap is small and well-understood rather than large and vague.

Three export mechanisms are configured in every SFT and DPO configuration:

- **LoRA adapters** written to the run's output workspace — tens of megabytes each, loadable over the shared frozen base. This is the artefact that makes many specialised models economically viable.
- `produce_merged_model: true` — deliberately flipped from false during the SFT work specifically to produce "a standalone merged model in addition to the LoRA adapters for easier deployment." Merged weights land alongside the adapters in the same run output workspace.
- **An Ollama export directory** configured in the run output — the only named serving runtime anywhere in the project, and the intended local-serving target.

Versioning and lineage exist, but informally. The platform stores Config, Logs, Eval Results and Artifacts per run, addressable by an opaque run identifier within the owning user's namespace — with the caveat, discovered during the 10 July cross-reference, that identifiers do not resolve across users. Job names carry version suffixes (document-description-sft-checkpoint-v3, website-summary-sft-3), and configuration files are versioned by filename. There is no model registry, no container image, and no CI/CD pipeline. `report_to: [none]` in every configuration means no experiment-tracking service was wired up either.

7.2 Intended path to production

The path below reflects what the project's own findings imply, sequenced so that each step de-risks the next. It is deliberately gated on the evaluation fix, because deploying against a metric that cannot discriminate would be worse than not deploying at all.

Step	Activity	Detail and rationale
1	Make the gate trustworthy	Surface held-out reward accuracy as a first-class, scored, gated metric on the platform. The per-record margins already exist in the evaluation logs; this is exposing what is there, not new instrumentation. No promotion decision should be automated until this is done.
2	Pin the environment	Pin the trainer library versions (the KTO blocker is a TRL regression) and add a minimal CI suite that trains a tiny model on each task type after every dependency change. This is the cheapest insurance in the entire plan.
3	Add a config pre-flight linter	Catch schema-forbidden keys, the early-stopping-requires-validation rule, and adapter-path resolution locally, before a GPU job is queued — failing in a second rather than nineteen seconds into a run that then blocks the shared GPU.
4	Register with a human gate	Promote a candidate only when it clears the discriminating metric, and only with explicit human approval. The team's stated guardrail is to automate the grind, not the judgment.
5	Serve	Merged model or base-plus-adapter via Ollama on the shared GPU. Serving multiple task adapters over one frozen base is the deployment topology the whole LoRA approach is designed for, and it is what makes a portfolio of small specialised models affordable.

Step	Activity	Detail and rationale
6	Orchestrate as a DAG	Chain data generation → SFT → DPO → evaluation → gated registration into one versioned, re-runnable workflow with dataset and configuration lineage, triggered automatically when new data lands.
7	Add a job queue	Auto-retry, automatic unloading of served models before training starts (a served model currently blocks new jobs), and ideally parallel or scheduled off-hours capacity to convert serial experimentation into concurrent sweeps.

Table 11 — Intended path to production.

One operational detail is worth flagging for whoever inherits this: on the current single-GPU platform, serving a model and training a model are mutually exclusive. New jobs fail to schedule while a model is being served. Any production deployment on this hardware therefore has to solve serving-versus-training contention before it can support continuous retraining — which is why the job queue with automatic model unloading appears in the plan rather than as an afterthought.

8. Limitations and Future Enhancements

8.1 What is incomplete, fragile, or out of scope

Incomplete against the original brief

- **The long-form wiki-style article was never built.** The 5 June requirement specified two outputs; one was delivered. The 300–400 word target also became ≤ 150 words. No artefact records a formal decision to descope, which is itself a process failure — the change should have been agreed and minuted with the sponsor.
- **PDF and uploaded-document ingestion was not implemented.** The requirement explicitly covered "a URL, documentation, or an uploaded file/PDF." Everything built handles crawled HTML documentation only. Multimodal PDFs with images were a stated stretch goal and remain untouched.
- **KTO is configured but has never run.** Datasets (200 train / 36 eval across 11 domains) and configurations are complete and validated. Khiron's attempts fail at roughly 13 seconds on a TRL KTOTrainer signature regression; Sean and Yash never ran the stage at all. The third alignment stage is therefore entirely unvalidated.
- **The three-way bake-off is unfinished.** Only two of the three contributors' evaluation sets were confirmed byte-identical. Khiron's runs were scored on his own splits, so his 74–79% band is indicative rather than strictly comparable to Sean's 84% and Yash's 93%. The team states this limitation itself.
- **No failure analysis on the best run.** The 93% result is the headline number and nobody has yet examined which held-out pairs it gets wrong or why.

Fragile

- **No version control, no dependency pinning, no CI.** There is no git repository, no requirements.txt, no container definition and no test suite anywhere in the project. Versioning is by filename. Three Python scripts carry the entire data pipeline and one of them — `build_pairs_v3.py` — hard-codes an absolute path from the authoring sandbox that will not resolve on any other machine.
- **The v3 datasets cannot be rebuilt from source.** The Gemini originals do survive — `backup_original/` holds 98 training and 30 evaluation pairs. But `build_pairs_v3.py` also requires `newdocs_A.jsonl`, `newdocs_B.jsonl` and `newdocs_C.jsonl`, and none of those three files exists in the delivered folder (only `newdocs_sources.jsonl` remains). The seeding was meant to guarantee reproducibility; the missing inputs defeat it.
- **The run ledger has known defects.** The Training Loss column mixes two different metrics across DPO rows (one row records a reward margin of 3.8144 where the actual loss was 0.045). Several run identifiers are duplicated, carry a stray carriage return, or are truncated; one does not resolve at all.
- **Platform telemetry is retained only for the most recent run.** Training-side reward accuracy is unavailable for four of Khiron's five completed DPO runs. Cross-run comparison had to be reconstructed by hand, and cannot be reconstructed at all for runs already overwritten.
- **The held-out set is weighted unevenly.** 17 of the 60 evaluation documents appear twice, so they carry roughly double weight. A de-duplicated 43-record variant exists but was not adopted as canonical, in order to preserve comparability with earlier runs.
- **Reference summaries were not fact-checked.** They are model-generated and validated by rule — word count, token cap, title presence, absence of navigation leakage — not verified line by line against the source documents.

Out of scope

- Licensing and terms-of-service review of the crawled vendor documentation; PII detection and redaction; NSFW and toxicity filtering; formal decontamination against a golden set. All are specified in the Corpus Formation Plan — a document scoped to a from-scratch pretraining corpus that was never built — and none were implemented for the fine-tuning corpus either.

- Any bias or fairness evaluation of the summarizer's outputs.
- A learned reward model and the PPO stage — both are gated on a dataset roughly fifty times larger than the one that exists.
- Production deployment, load testing, latency measurement, and cost-per-inference analysis.

8.2 The structural ceiling

The team's own assessment — and the evidence supports it — is that progress is no longer limited by tuning skill. Three constraints form a reinforcing cycle:

Why the bottlenecks reinforce each other Manual data preparation keeps datasets small. Small datasets make DPO memorise and cap generalisation, so lever changes move the training curves but not held-out quality. The saturated evaluation metric then hides whether any change actually helped. And manual, serial experimentation on a single GPU makes every attempt to break the cycle slow and expensive. Breaking any one link helps, but the leverage sits at two roots: a discriminating metric, so that progress becomes visible at all; and automated data generation at scale, so that the overfitting pressure eases. That is precisely why the roadmap sequences those two first — and why more hyperparameter tuning would be the least valuable thing a successor team could do.	
Constraint	Evidence and magnitude
Dataset size (dominant)	~200 preference pairs against a stated sweet spot of 1,000–2,000; a learned reward model needs 10,000+. Across the entire lever sweep, held-out accuracy moved only within 0.78–0.84 — the levers changed training dynamics but not generalisation.
Evaluation quality	Only two built-in scorers exist (exact match, embedding similarity). For free-text summaries the embedding score saturates and is dominated by the SFT base. The discriminating metric is derivable but must be parsed out of the logs by hand.
Compute and throughput	One shared 16 GB GPU, one job at a time; new jobs fail to schedule if a job is running or a model is being served. Caps model size at roughly 3B, forces batch size 1, and turns a parallel sweep into a serial queue.
Manual experimentation	Every run is hand-driven: author config, launch, wait, read telemetry, diagnose, change one lever, relaunch. Slow, hard to reproduce, and hostile to unattended overnight sweeps.
Manual data preparation	Every pair hand-built: crawl, clean, cap, generate a good and a deliberately degraded summary, validate. This is the practical reason the datasets stayed small.
Platform fragility	Silent schema rejections, telemetry retained for the latest run only, and a library regression blocking KTO. Each avoidable with validation and pinning; each cost real engineering time.
Team availability	Recorded as a recurring blocker across four consecutive weekly updates, alongside intermittent platform downtime.

Table 12 — The structural constraints, ordered by impact.

8.3 Concrete next steps for a future team

The following is ordered so that each phase de-risks the next. A successor team should resist the temptation to start at phase two.

Phase	Focus	Concrete next steps
1	Foundations — trustworthy signal	Surface held-out reward accuracy as the scored, gated metric (the data is already logged). Pin TRL and add a CI job that trains a tiny model on each task type after every dependency change — this alone unblocks KTO. Write a config pre-flight linter for schema-forbidden keys, the early-stopping-needs-validation rule, and adapter-path resolution. Retain telemetry for all runs, not just the latest. Fix the two known defects in the run ledger. Recover or

Phase	Focus	Concrete next steps
		regenerate the three missing newdocs_A/B/C.jsonl inputs so the v3 datasets are reproducible.
2	Data scale — break the dominant bottleneck	Build the data-sourcing agent: discover and fetch sources for a named domain, strip code and navigation, cap to budget, and generate SFT demonstrations, DPO triples and KTO rows at the thousands-of-pairs scale DPO actually needs. Build in the quality gates that are currently manual — word-count and reasoning-tag checks, de-duplication, guaranteed train/eval disjointness, and an automated quality filter for noisy or mislabelled pairs. Add the licensing, PII and decontamination gates that were specified but never implemented. Optionally go on-policy: sample candidate summaries from the current SFT checkpoint and label those, which yields higher-signal pairs than hand-built ones.
3	Automation — remove the grind	Build the auto-tuning agent: parse reward accuracy, margin, loss and KL from the telemetry plus the held-out evaluation; classify each run as overfit, underfit or healthy using the decision rules this project already validated; change one lever per run so effects stay attributable; orchestrate sweeps ranked on held-out reward accuracy and stop when accuracy lands in the 0.6–0.9 acceptance window. Add a job queue with auto-retry and automatic unloading of served models. Keep a human approval gate before registration.
4	Close the loop	Once data volume passes roughly 1,000 pairs, train a reward model as a learned, reusable scorer feeding both evaluation and, later, PPO for on-policy reinforcement learning. Chain the whole pipeline into a versioned DAG with lineage, re-triggered when new data lands.

Table 13 — Four-phase roadmap. Every stage in the proposed pipeline corresponds to a task this team performed manually; what is missing is orchestration and a trustworthy metric, not new science.

Guardrails a successor team should build in from the start

- **Optimising the wrong metric.** An auto-tuner faithfully maximises whatever it is pointed at. Aimed at the saturated pass-rate — which ranked the underfit model above the winner — it would confidently make the pipeline worse. Gate the tuner on the discriminating held-out metric, never on the default.
- **Reward hacking.** Once a learned reward model enters the loop, the policy can find shortcuts that score well without being good. Combine the learned reward with deterministic checks and validate against a separate held-out reward model.
- **Over-automation.** Keep a human approval gate before registration and deployment.
- **Data drift and provenance.** An agent generating data at scale can quietly introduce bias or duplication. Track provenance automatically, de-duplicate, and spot-check generated data by hand periodically.
- **Silent platform regressions.** As the KTO bug showed, a library upgrade can break a task type without warning. Version pinning plus a small CI suite is the answer, and it belongs in phase one.

9. Conclusion

The question the team set out to answer was whether a very small model, trained on a single shared GPU through a repeatable process, could perform a real job reliably. The answer is yes, with a qualification that turned out to be more valuable than the answer itself.

On the narrow problem: **solved**. The supervised stage is complete and dependable, passing 100% of held-out summaries on the largest evaluation set and 24–25 of 25 on the standard one. The preference stage is tuned to a defensible production configuration, promoted to `dpo_config_FINAL.yaml`, with training metrics squarely in the healthy band — and the strongest run in the programme reached 93% held-out reward accuracy with no train-versus-held-out gap. Two hard failures, an out-of-memory crash and a reasoning leak, were root-caused and permanently eliminated. A model within a rounding error of the sponsor's size constraint does the job it was asked to do.

On the sponsor's original brief: **partially delivered**. The short summary was built; the long-form wiki-style article and PDF ingestion were not, and the summary length target moved from 300–400 words to 150. Judged strictly against the 5 June requirement, roughly half the specified output surface exists. Judged against what the sponsor said they actually wanted — a training capability that can produce small specialised models at scale — the narrow scope was the right trade, because it is what made the pipeline cheap enough to iterate on 47 times and to instrument properly.

On the deeper question: **the project found the real constraint, which is the most useful thing a proof of concept can do**. It is not tuning skill. Four levers were tested rigorously across six controlled runs and only two moved anything, while held-out quality stayed flat at 0.78–0.84 across the entire sweep. Two runs with identical hyperparameters produced 63% and 93% depending only on the data and checkpoint underneath them. And the scoreboard everyone was optimising against could not tell a good model from a bad one — it ranked the clearly underfit baseline above the tuned winner. The binding constraints are data scale, evaluation quality, and a manual serial workflow, in that order.

Two things give that conclusion weight rather than leaving it as an assertion. The first is that the replacement metric costs nothing to adopt: held-out reward accuracy is computable from per-record margins the platform already writes to disk, and switching to it turned a flat line into a thirty-point spread across the same runs and the same logs. The second is that the comparison underpinning the headline result was verified rather than assumed — two contributors' evaluation files were confirmed identical, prompt by prompt and answer by answer, before their scores were placed side by side.

What the team hands over, then, is not primarily a summarizer. It is a mapped configuration space, an evidence-backed list of platform defects, a validated set of diagnostic decision rules, and a sequenced plan for automating a loop that has already been proven by hand. The next chapter of value here is not a better hyperparameter. It is a better measurement, more data, and the automation to exploit both — so that the same pipeline that trained one summarizer can train specialised models for whatever task comes next.

Was the problem solved?

Yes for the technical question — a 0.6B model trained on one shared GPU performs the summarization task reliably, and the process that produced it is documented, repeatable and evidenced across 47 runs.

Partially for the sponsor's full brief — one of two specified outputs was delivered, at a shorter length than requested, and document/PDF ingestion was not built.

And more than was asked for on the underlying question the sponsor actually cares about: the project established, with evidence, exactly what stands between this pipeline and a scalable training capability — and cost the sponsor almost nothing to find out.

Appendix — Key Code Files Overview

The entire data pipeline is carried by three Python files, all located in `Model Training Planning/DP0/`. There is no other executable code in the project: no notebooks, no shell scripts, no test suite, no repository. The scripts use only the Python standard library and are seeded for reproducibility. Training behaviour itself is expressed declaratively in YAML configurations submitted to the platform, which are catalogued after the scripts.

A.1 Python scripts

`build_dpo_dataset.py` — 151 lines

Purpose. The deterministic scaffold for the crawl-to-JSONL pipeline. It parses the two numbered URL lists out of the README, cleans page text, builds the prompt field, applies the length cap, and emits prompt-only rows with empty chosen and rejected slots to be filled interactively.

Inputs. `README.md` (the "Training set" and "Evaluation set" numbered lists — these README files are the pipeline's input contract); optionally `--textdir`, a directory of pre-cleaned `.txt` files named by URL slug.

Outputs. `dpo_train_to_fill.jsonl` and `dpo_eval_to_fill.jsonl`, each row carrying `_source_url`, prompt, and empty chosen/rejected fields.

Key constants and behaviour. `PROMPT_PREFIX` = "Summarize the following document, capturing the key points, in 150 words or less."; `MAX_PROMPT_CHARS` = `3700 * 4` ($\approx 3,700$ tokens at roughly four characters per token). Cleaning strips fenced and inline code, blank lines, and short non-alphanumeric lines. Defines `strip_think()` to remove leaked reasoning blocks. Two modes: `--mode list` prints parsed URLs; `--mode scaffold` writes the fill files.

Note. The docstring is where the AI-assistance disclosure originates: "in this project the crawl + summary-writing is performed interactively (Claude fetches and cleans each page, then writes the chosen/rejected text). This script is the deterministic scaffold for the mechanical parts so the pipeline is repeatable whenever the README URL lists change."

`build_pairs_v2.py` — 147 lines

Purpose. The preference-pair regenerator, and the most technically interesting file in the project. It keeps each accurate chosen summary (after `strip_think`) and synthetically rebuilds the rejected summary as exactly one controlled, *labelled* defect — so every pair's failure mode is known and countable rather than merely "worse."

The six defect generators and their weights:

Generator	Weight	What it does
<code>make_factual_error</code>	30	Mutates integers; flips GET→POST, required→optional, UUID→integer ID, bearer token→API key query parameter.
<code>make_wrong_scope</code>	22	Substitutes a neighbouring document's summary — plausible prose describing the wrong thing.
<code>make_vague</code>	18	Replaces specifics with generic text that could describe almost any document.
<code>make_key_omission</code>	14	Retains only the first third of the sentences, dropping the substantive detail.
<code>make_verbose_pad</code>	8	Injects canned filler sentences, pushing the summary well over the length limit.
<code>make_think_leak</code>	8	Prepends a canned <code><think></code> reasoning block — the exact failure mode observed in early SFT runs.

Inputs. `--indir` containing `dpo_train.jsonl` and `dpo_eval.jsonl`; `--train-variants` and `--eval-variants` (default 2 each). Seeded with `random.seed(42)`.

Outputs. Rebuilt `dpo_train.jsonl` and `dpo_eval.jsonl`; `dpo_train_labels.jsonl` and `dpo_eval_labels.jsonl` carrying {defect_type}; and `data_quality_report.txt` with pair counts, chosen/rejected word medians, the percentage of rejections

longer than their chosen counterpart, the percentage containing <think>, and the realised defect distribution. Note that the report delivered in the project folder is the v3-format one written by the assembler below.

Measured output on the final 200-pair training set, as regenerated by build_pairs_v3.py: chosen median 129 words (max 147); rejected median 128; rejected longer than chosen in 81 of 200 (40%); <think> present in 14 of 200 (7%). Realised defect distribution: factual_error 64, wrong_scope 45, vague_generic 35, key_omission 28, think_leak 14, verbose_pad 14.

build_pairs_v3.py — 91 lines

Purpose. The domain-diversification assembler. It imports build_pairs_v2 as a library and blends two source families into the final datasets, holding out four new documents entirely to prevent leakage.

Composition it produces. Training set = 30 new technical documents × 3 variants (90 pairs) + 55 Gemini documents × 2 variants (110 pairs) = 200. Evaluation set = 4 new × 2 (8) + 13 Gemini × 2 (26) = 34. It also trims each chosen summary to ≤150 words at sentence boundaries, joint-shuffles pairs with their labels, and tags every label with a domain field. Seeded with `random.seed(7)`.

Inputs. `backup_original/dpo_train.jsonl` (98 unique Gemini pairs) and `backup_original/dpo_eval.jsonl` (30 unique), plus `newdocs_A/B/C.jsonl`.

Outputs. `v3data/dpo_train.jsonl`, `v3data/dpo_eval.jsonl`, the two label files carrying {defect_type, domain}, and `v3data/data_quality_report.txt`.

Two defects worth flagging to a successor team

Line 37 hard-codes an absolute path from the authoring sandbox that will not resolve on any other machine. `build_dpo_dataset.py` carries a related defect: it hard-codes the input filename `README.md`, which does not match the delivered `dpo_README.md`. Both must be parameterised before the scripts can run anywhere else.

The Gemini originals in `backup_original/` are present (98 train / 30 eval), but the three `newdocs_A/B/C.jsonl` inputs the script requires are absent from the delivered folder — only `newdocs_sources.jsonl` remains. Until they are recovered, the v3 datasets cannot be rebuilt from source, which defeats the purpose of the seeding.

A.2 Configuration files

Training behaviour is declarative. These YAML files are effectively the project's source code for everything that happens on the GPU.

File	Lines	Role
SFT Base/sft_config.yaml	73	The production SFT configuration. task_type causal_lm_sft; LoRA r16/α32/dropout 0.05; lr 2e-4; 3 epochs; max_seq_length 4096; validation_split 0.2; paged_adamw_8bit; bf16; gradient checkpointing; early stopping on eval_loss with patience 3; assistant_prefix "<think></think>" to suppress reasoning.
SFT Base/sft_config_v2.yaml	—	Superseded SFT variant retained for provenance.
DPO/dpo_config.yaml	154	Heavily annotated working DPO configuration; documents the Pydantic extra="forbid" schema rejections encountered on 3 July.
DPO/dpo_config_FINAL.yaml	110	The promoted production configuration. beta 0.10; sigmoid loss; r8/α16/dropout 0.10; 2 epochs; lr 5e-6; weight decay 0.01; gradient_accumulation_steps 8; validation_split 0.1; initial_adapter_path pointing at the SFT adapters; produce_merged_model true; Ollama export configured. Header records the winning metrics: reward accuracy 0.7500, margin +0.4401.
DPO/dpo_config_clean.yaml, _v2.yaml	—	Intermediate revisions retained for lineage.
DPO/experiments/dpo_config_run1-	6 files	One file per lever experiment — the mechanism that made the single-variable

File	Lines	Role
6.yaml		discipline enforceable rather than aspirational.
KTO/kto_config.yaml, _v2, _v2_clean	3 files	Complete and validated, but every run fails at ~13 seconds on the TRL KTOTrainer regression.
PPO/ppo_config.yaml	—	Prepared but unusable until a learned reward model exists.

A.3 Data manifests and datasets

File	Size	Role
SFT Base/sft_README.md	152 lines	The SFT input contract: 100 training and 25 evaluation URLs, plus the six documented dataset iterations and the five-stage pipeline.
DPO/dpo_README.md	180 lines	The DPO input contract: 130 URLs across Gemini and Firecrawl, plus the workflow instructions used to drive the AI-assisted crawl.
KTO/kto_README.md	~200 lines	118 URLs across 11 vendor domains, explicitly non-overlapping with the SFT and DPO sources.
DPO/shared_dpo_eval_MANIFEST.md	42 lines	The freeze instruction and the byte-identity verification record for the canonical 60-record held-out set.
DPO/data_quality_report.txt	9 lines	Generated dataset statistics: pair counts, domain mix, word medians, longer/<think> percentages and the realised defect distribution.
DPO/dpo_train.jsonl	200 rows	The production preference training set.
DPO/shared_dpo_eval.jsonl	60 rows	The canonical held-out set, 43 unique documents, 26 of them out-of-domain.
DPO/shared_dpo_eval_unique43.jsonl	43 rows	De-duplicated variant addressing the double-weighting issue; not adopted as canonical.
SFT Base/sft_train_v3.jsonl / sft_eval_v3.jsonl	100 / 25	The current SFT datasets across OpenAI, Llama and DeepSeek documentation.
Model_Runs_Updated.xlsx	4 tabs, 47 runs	The system of record: Overview, SFT, DPO and KTO tabs with full hyperparameters, metrics and verdicts per run.

Sources. SFT_DPO_Training_Report_EXPO.docx; Model_Runs_Updated.xlsx (Overview / SFT / DPO / KTO tabs); Capstone Project – Outliers charter; Internal Sync 5 June 2026; weekly progress reports of 1 June, 5 July (×2), 8 July, 10 July and 13 July 2026; SFT and DPO dataset and configuration reports; DPO Overfitting Analysis and Remediation; shared_dpo_eval_MANIFEST.md; data_quality_report.txt; the three build scripts and all SFT, DPO, KTO and PPO configuration files; Custom Model Feasibility Plan and Corpus Formation Plan; Model Runs Log Cross-Reference (10 July 2026); AI Capstone 12-minute speaker script and Team Outliers talk track; expo board design materials.

Note on evidence. Where a section of the requested outline could not be substantiated from project artefacts — formal sprint ceremonies, a code repository, CI configuration, a bias evaluation, or a production deployment — this report says so explicitly rather than describing intent as if it were implementation.